tut_stdtypes/read_songdata_1.rb
```
Song = Struct.new(:title, :name, :length)

songs = File.readlines("code/tut_stdtypes/songdata", chomp: true).map do |line|
  _file, length, name, title = line.chomp.split(/\s*\|\s*/)
  Song.new(title, name, length)
end
puts songs[1]
```

produces:

```
#<struct Song title="Wonderful World", name="Louis    Armstrong", length="2:58">
```

We're using readlines and map here to convert each line of the input file into its own Song.

Unfortunately, whoever created the original file entered the artists' names in columns, so some of them contain extra spaces that we'd better remove before we go much further. We have many ways of doing this, but probably the simplest is String#squeeze, which trims runs of repeated characters.

tut_stdtypes/read_songdata_2.rb
```
Song = Struct.new(:title, :name, :length)

songs = File.readlines("code/tut_stdtypes/songdata", chomp: true).map do |line|
  _file, length, name, title = line.chomp.split(/\s*\|\s*/)
  Song.new(title, name.squeeze(" "), length)
end
puts songs[1]
```

produces:

```
#<struct Song title="Wonderful World", name="Louis Armstrong", length="2:58">
```

Finally, we have the minor matter of the time format: the file says 2:58, and we want the number of seconds, 178. We could use split again, this time splitting the time field around the colon character:

```
"2:58".split(":") # => ["2", "58"]
```

Instead, we'll use a related method. String#scan is similar to split in that it breaks a string into chunks based on a pattern. However, unlike split, with scan you specify the pattern that you want the chunks to match. In this case, we want to match one or more digits for both the minutes and seconds components. The pattern for one or more digits is /\d+/, then we convert the resulting minutes and seconds to length in seconds.

tut_stdtypes/read_songdata_3.rb
```
Song = Struct.new(:title, :name, :length)

songs = File.readlines("code/tut_stdtypes/songdata", chomp: true).map do |line|
  _file, length, name, title = line.chomp.split(/\s*\|\s*/)
  minutes, secs = length.scan(/\d+/)
  Song.new(title, name.squeeze(" "), minutes.to_i * 60 + secs.to_i)
end
puts songs[1]
```

produces:

```
#<struct Song title="Wonderful World", name="Louis Armstrong", length=178>
```

We could spend the next fifty pages looking at all the methods in class String. However, let's move on instead to look at a simpler data type: the range.